
Testing

Lukas Holik

lukasholik@cs.aau.dk

(based on slides of Gabriela Montoya, thanks)

Motivation

- Systematic testing to ensure correctness is necessary for every larger project.
- Testing is done at multiple levels - unit, integration, system.
- Exhaustive testing is usually impossible, various metrics are used to assess the quality of tests, e.g. code coverage.
- Tests can be chosen based on specifications or on code.
 - Specification-based or functional or black-box testing,
 - e.g., based on the method specification.
 - Code-based or structural or white-box testing,
 - e.g., based on the execution of statements/branches.

Black box testing

find

```
public Course find(Stringu code)
```

an instance method that finds a course in the catalog

Parameters:

code - the code of the course to find in the catalog

Returns:

Returns the course if it is in the catalog or null otherwise

The test set must at least verify that:

- The right course is returned if the course is in the catalog.

```
package dk.aau.test;

import dk.aau.adt.*;

public class CatalogTest {

    public static void main(String[] args) {
        Catalog catalog = new Catalog();
        String code = "OOP";
        String title = "Object-oriented programming";
        Course c = new Course(code, title);
        catalog.addElem(c);

        assert c.equals(catalog.find(code));
        System.out.println("Done");
    }
}
```

Black box testing

find

```
public Course find(StringⓂ code)
```

an instance method that finds a course in the catalog

Parameters:

code - the code of the course to find in the catalog

Returns:

Returns the course if it is in the catalog or null otherwise

The test set must at least verify that:

- The right course is returned if the course is in the catalog,
- null is returned if the code is not register in the catalog,
- null is returned if the code is null.

```
public static void main(String[] args) {  
    Catalog catalog = new Catalog();  
    String code = "OOP";  
  
    assert catalog.find(code)==null;  
    System.out.println("Done");  
}
```

```
public static void main(String[] args) {  
    Catalog catalog = new Catalog();  
    String code = null;  
  
    assert catalog.find(code)==null;  
    System.out.println("Done");  
}
```

Black box testing

find

```
public Course find(Stringu code)
```

an instance method that finds a course in the catalog

Parameters:

code - the code of the course to find in the catalog

Returns:

Returns the course if it is in the catalog or null otherwise

The test set must at least verify that:

- The right course is returned if the course is in the catalog.

```
package dk.aau.test;
```

```
import org.junit.jupiter.api.*;  
import dk.aau.adt.*;
```

```
public class CatalogTest {
```

```
    private Catalog catalog;
```

@BeforeEach

```
void init() {  
    catalog = new Catalog();  
}
```

@Test

```
void findRegisteredCourse() {  
    String code = "OOP";  
    String title = "Object-oriented programming";  
    Course c = new Course(code, title);  
    catalog.addElem(c);  
    Assertions.assertEquals(c, catalog.find(code));  
}  
}
```

Black box testing

find

```
public Course find(String code)
```

an instance method that finds a course in the catalog

Parameters:

code - the code of the course to find in the catalog

Returns:

Returns the course if it is in the catalog or null otherwise

The test set must at least verify that:

- The right course is returned if the course is in the catalog,
- null is returned if the code is not register in the catalog,
- null is returned if the code is null.

```
...
public class CatalogTest {

    private Catalog catalog;

    @BeforeEach
    void init() {
        catalog = new Catalog();
    }
    ...

    @Test
    void findNonRegisteredCourse() {
        String code = "OOP";
        Assertions.assertNull(catalog.find(code));
    }

    @Test
    void findNull() {
        String code = null;
        Assertions.assertNull(catalog.find(code));
    }
}
```

White box testing

```
import java.util.ArrayList;

class Catalog {
    private ArrayList<Course> elems;
    ...

    public Course find(String code) {
        Course res = null;
        for (Course c : this.elems) {
            if (c.getCode().equals(code)) {
                res = c.clone();
            }
        }
        return res;
    }
}
```

Chosen **coverage criterion**:

Test each "category" of execution paths.

Categories as combinations of the following cases:

for - considered cases:

1. Empty collection collection.
2. Single element in the collection.
3. Multiple elements in the collection.

if - considered cases:

- A. The expression evaluates as true.
- B. The expression evaluates as false.

Possible categories of paths:

- 1. (findNonRegisteredCourse)
- 2A. (findRegisteredCourse)
- 2B.
- 3A.
- 3B.

Case 2B

```
@Test
void findSingletonFalse() {
    String code1 = "OOP";
    String title = "Object-oriented
programming";
    Course c = new Course(code1, title);
    String code2 = "SU";
    catalog.addElem(c);
    Assertions.assertNull(catalog.find(code2));
}
```

```
public Course find(String code) {
    Course res = null;
    for (Course c : this.ems) {
        if (c.getCode().equals(code)) {
            res = c.clone();
        }
    }
    return res;
}
```

for - considered cases:

1. Empty collection collection.
2. Single element in the collection.
3. Multiple elements in the collection.

if - considered cases:

- A. The expression evaluates as true.
- B. The expression evaluates as false.

Which case does anotherTestMethod correspond to? 3A, or to 3B ?

```
@Test
void anotherTestMethod() {
    String code1 = "OOP";
    String title1 = "Object-oriented programming";
    Course c1 = new Course(code1, title1);
    String code2 = "SU";
    String title2 = "Systems Development";
    Course c2 = new Course(code2, title2);
    String code3 = "DEB";
    catalog.addElem(c1);
    catalog.addElem(c2);
    Assertions.assertNull(catalog.find(code3));
}
```

```
public Course find(String code) {
    Course res = null;
    for (Course c : this.ems) {
        if (c.getCode().equals(code)) {
            res = c.clone();
        }
    }
    return res;
}
```

for - considered cases:

1. Empty collection collection.
2. Single element in the collection.
3. Multiple elements in the collection.

if - considered cases:

- A. The expression evaluates as true.
- B. The expression evaluates as false.

Where bugs typically come from?

- unexpected inputs
- a method call that throws an unexpected exception
- a malfunctioning library
- mismatching parameters
- incorrect protocols
- unexpected sequence of actions
-
- places no one knew existed
- places considered so harmless that no one bothered to think about them

How to choose inputs to be tested?

- Input values:
 - individual variables: choose values at or around boundaries
 - date to book a room: now, yesterday, tomorrow, non-existent date, ...
 - array index: just before/after the array range (-1,0,length-1, length)
 - several variables: all combinations of their values (cross product)

Input values for different types of scenarios

```
public static void loadCourses(String fileName) throws FileNotFoundException,  
                                                NoSuchElementException {  
    FileReader fr = new FileReader(fileName);  
    Scanner s = new Scanner(fr);  
    while(s.hasNextLine()) {  
        String line = s.nextLine();  
        StringTokenizer st = new StringTokenizer(line, ",");  
        String code = st.nextToken();  
        String title = st.nextToken();  
        Course c = new Course(code, title);  
    }  
}
```



OOP, Object-oriented programming
SU, Systems Development
DEB, Design and evaluation of user interfaces

OOP
SU
DEB

OOP Object-oriented programming
SU Systems Development
DEB Design and evaluation of user interfaces

OOP, Object-oriented programming, SU
Systems Development
DEB
Design and evaluation of user interfaces

Testing exceptions

```
public static void loadCourses(String fileName) throws FileNotFoundException, NoSuchElementException {  
    FileReader fr = new FileReader(fileName);  
    Scanner s = new Scanner(fr);  
    while(s.hasNextLine()) {  
        String line = s.nextLine();  
        StringTokenizer st = new StringTokenizer(line, ",");  
        String code = st.nextToken();  
        String title = st.nextToken();  
        Course c = new Course(code, title);  
    }  
}
```

```
@Test  
void loadCoursesExistingFileCorrectFormat() {  
    String fileName = "input\\courses.csv";  
    Executable e = () -> { Course.loadCourses(fileName); };  
    Assertions.assertDoesNotThrow(e);  
}
```

```
@Test  
void loadCoursesNonExistingFile() {  
    String fileName = "input\\nonExistingFile.txt";  
    Executable e = () -> { Course.loadCourses(fileName); };  
    Assertions.assertThrows(java.io.FileNotFoundException.class, e);  
}
```

OOP, Object-oriented programming
SU, Systems Development
DEB, Design and evaluation of user interfaces



org.junit.jupiter.api.function.Executable

```
@FunctionalInterface  
  
public interface Executable {  
    void execute();  
}
```

An object, its type is a new nameless class that implements interface Executable, that is, the class implements a method execute, that runs the Body.

```
String fileName = "input\\courses.tsv";  
Executable e = () -> { Course.loadCourses(fileName); };
```

Lambda expression

Body of the **execute** method

Parameters of the **execute** method (arity 0)

Testing more exceptions

```
public static void loadCourses(String fileName) throws FileNotFoundException, NoSuchElementException {  
    FileReader fr = new FileReader(fileName);  
    Scanner s = new Scanner(fr);  
    while(s.hasNextLine()) {  
        String line = s.nextLine();  
        StringTokenizer st = new StringTokenizer(line, ",");  
        String code = st.nextToken();  
        String title = st.nextToken();  
        Course c = new Course(code, title);  
    }  
}
```

```
@Test  
void loadCoursesExistingFileIncorrectFormat() {  
    String fileName = "input\\courses2.csv";  
    Executable e = () -> { Course.loadCourses(fileName); };  
    Assertions.assertThrows(java.util.NoSuchElementException.class, e);  
}
```

OOP Object-oriented programming
SU Systems Development
DEB Design and evaluation of user interfaces

```
@Test  
void loadCoursesExistingFileIncorrectFormat2() {  
    String fileName = "input\\courses.tsv";  
    Executable e = () -> { Course.loadCourses(fileName); };  
    Assertions.assertThrows(java.util.NoSuchElementException.class, e);  
}
```

OOP, Object-oriented programming, SU
Systems Development
DEB
Design and evaluation of user interfaces

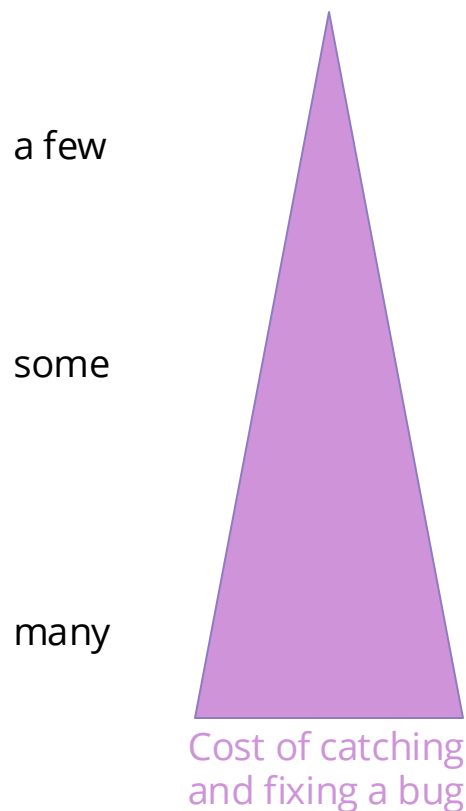
Coverage

- Ideally,
 - test all sequences of statements = execution paths (**execution path coverage**)
 - test all sequences of inputs (**input sequence coverage**)
- Full execution path and input sequence coverage are unattainable.
- Various coverage criteria that approximate them are used.

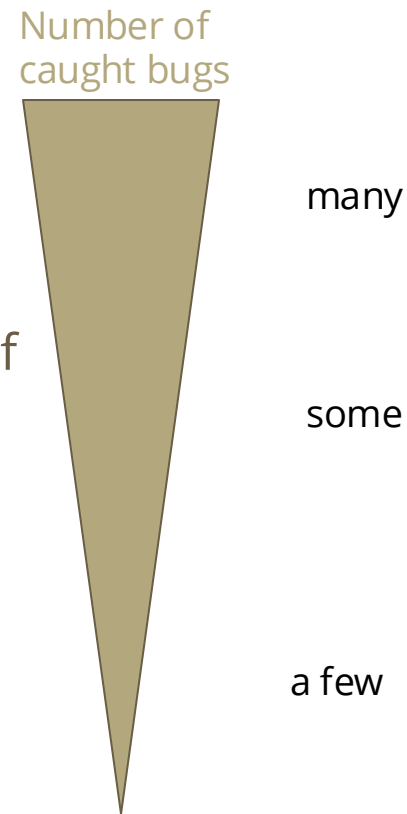
Practical coverage criteria

- For execution paths, select a set of tests that
 - executes each statement at least once → **statement coverage**
 - executes each possible branch at least once → **branch coverage**
 - initializes and uses each object/data struct → **object/data struct. coverage**
- For input sequence, select a set of tests that
 - contains each possible input → **exhaustive testing**
 - contains the sequences that a typical user will use → **operation profile testing/representative test suite**

Levels of testing



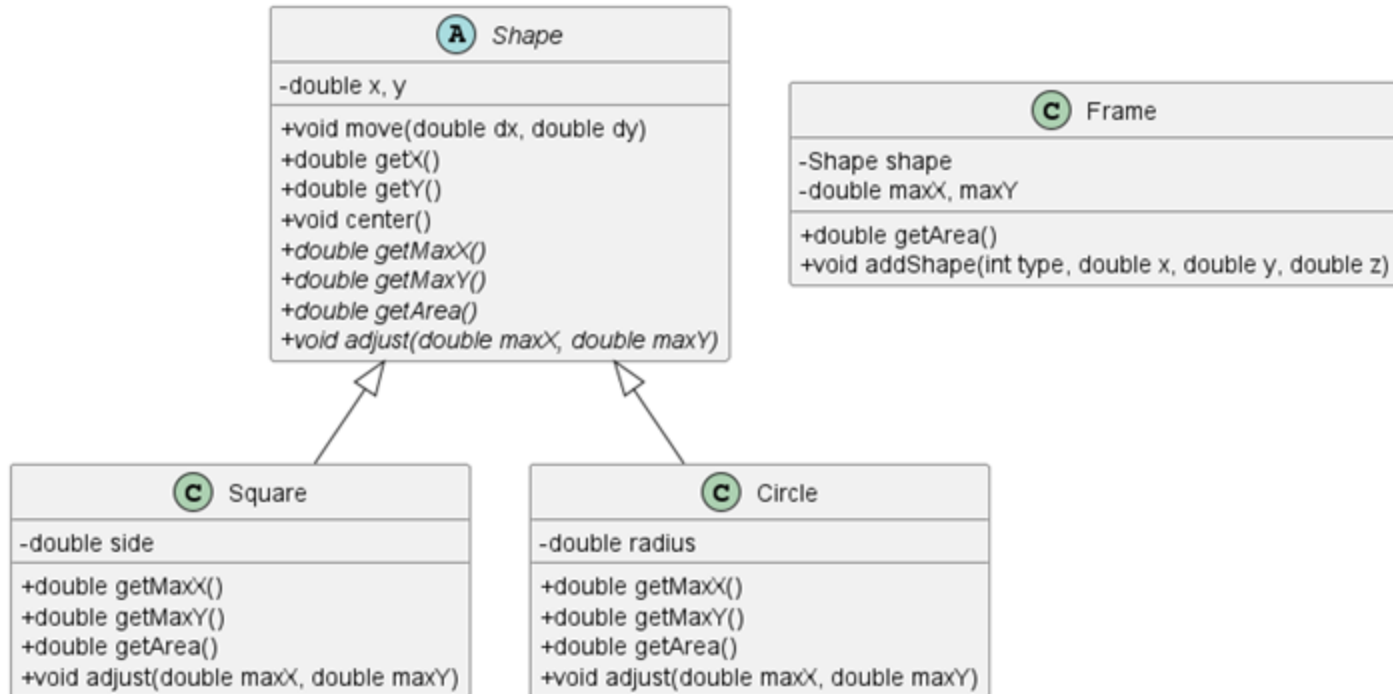
- **Unit testing:** validation of program units.
- **Integration testing:** validation of the interfaces between the program units.
- **System testing:** end-to-end validation of a deliverable product.



Integration testing

- Ideally, the interactions between units are exercised as completely as possible.
- Testing of pairs of units bounds its complexity.
- Intra-class (between classes) and inter-class (inside one class).

Example



Difficulty of testing object-oriented programs

- Inheritance and polymorphism increase the difficulty of finding faults

```
public abstract class Shape {  
    private double x, y;  
  
    public Shape(double x, double y) {  
        this.x = x;  
        this.y = y;  
    }  
    public void center() {  
        ...  
    }  
    ...  
}
```

```
public class Square extends Shape {  
    private double side;  
  
    public Square(double x, double y, double s) {  
        super(x, y);  
        this.side = s;  
    }  
    @Override  
    public void adjust(double maxX, double maxY) {  
        center();  
        radius = Math.min(radius, Math.min(maxX, maxY));  
    }  
    ...  
}
```

java Frame 10 1 0 0 15

java Frame 10 2 0 0 25

Difficulty of testing object-oriented programs

- The lifetime of an object can overlap with several units.

```
class Frame {  
  
    private Shape shape;  
    private double maxX, maxY;  
    ...  
    public static void main(String[] args) {  
        double max = Double.parseDouble(args[0]);  
        int type = Integer.parseInt(args[1]);  
        double x = Double.parseDouble(args[2]);  
        double y = Double.parseDouble(args[3]);  
        double z = Double.parseDouble(args[4]);  
  
        Frame f = new Frame(max);  
        f.addShape(type, x, y, z);  
        System.out.println(f+"\narea: "+f.getArea());  
    }  
}
```

java Frame 10 1 0 0 15

java Frame 10 2 0 0 25

All-bindings coverage criteria

- Make sure that inherited methods are tested.
- Make sure that the methods work well when late binding is used for the possible subtypes.

```
public void addShape(int type, double x, double y, double z) {  
    if (type==1) {  
        shape = new Circle(x, y, z);  
    } else {  
        shape = new Square(x, y, z);  
    }  
  
    if (shape.getMaxX()>this.maxX || shape.getMaxY() > this.maxY) {  
        shape.adjust(this.maxX, this.maxY);  
    }  
}
```

java Frame 10 1 0 0 15
java Frame 10 2 0 0 25

All-du-pairs (definition-use pairs)

- Make sure that every object that is created or updated is tested for all their possible uses through the program.

```
9. class Frame {  
10.     private Shape shape;  
11.     private double maxX, maxY;  
    ...  
44.     public static void main(String[] args) {  
45.         double max = Double.parseDouble(args[0]);  
46.         int type = Integer.parseInt(args[1]);  
47.         double x = Double.parseDouble(args[2]);  
48.         double y = Double.parseDouble(args[3]);  
49.         double z = Double.parseDouble(args[4]);  
  
50.         Frame f = new Frame(max);  
51.         f.addShape(type, x, y, z);  
52.         System.out.println(f+"\narea: "+f.getArea());  
53.     }  
54. }
```



```
class Frame {
    private Shape shape;
    private double maxX, maxY;
    ...
    public double getArea() {
        double area = 4*maxX*maxY;
        area -= shape.getArea();
        return area;
    }
}
```

	getArea	getArea	getArea
def			
use	maxX, maxY, shape	side	radius

	adjust	adjust
def	x, y, side	x, y, radius
use	side	radius

** constructors
(define everything)*

```
public class Square extends Shape {
    private double side;
    ...
    @Override
    public double getArea() {
        return side * side;
    }
    @Override
    public void adjust(double maxX, double maxY) {
        center();
        side = Math.min(side, Math.min(2*maxX, 2*maxY));
    }
}
```

```
public class Circle extends Shape {

    private double radius;
    ...
    @Override
    public double getArea() {
        return Math.PI * Math.pow(radius, 2);
    }
    @Override
    public void adjust(double maxX, double maxY) {
        center();
        radius = Math.min(radius, Math.min(maxX, maxY));
    }
}
```

Where is the state of Frame used, where is it defined?

```
9. class Frame {  
  
10.     private Shape shape;  
11.     private double maxX, maxY;  
    ...  
25.     public void addShape(int type, double x, double y, double z) {  
26.         if (type==1) {  
27.             shape = new Circle(x, y, z);  
28.         } else {  
29.             shape = new Square(x, y, z);  
30.         }  
31.  
32.         if (shape.getMaxX()>this.maxX || shape.getMaxY() > this.maxY) {  
33.             shape.adjust(this.maxX, this.maxY);  
34.         }  
35.     }
```

java Frame 10 1 0 0 15

```
@Test
void test1() {
    Frame frame = new Frame(10);
    frame.addShape(1, 0, 0, 15);
    double area = frame.getArea();
    Assertions.assertTrue(area >= 0);
}
```

java Frame 10 2 0 0 25

```
@Test
void test2() {
    Frame frame = new Frame(10);
    frame.addShape(2, 0, 0, 25);
    double area = frame.getArea();
    Assertions.assertTrue(area >= 0);
}
```

```
9. class Frame {
10.     private Shape shape;
11.     private double maxX, maxY;
12.
13.     ...
14.
15.     public void addShape(int type, double x, double y, double z) {
16.         if (type==1) {
17.             shape = new Circle(x, y, z);
18.         } else {
19.             shape = new Square(x, y, z);
20.         }
21.
22.         if (shape.getMaxX() > this.maxX || shape.getMaxY() > this.maxY) {
23.             shape.adjust(this.maxX, this.maxY);
24.         }
25.     }
26.
27.     ...
28.
29.     public static void main(String[] args) {
30.         double max = Double.parseDouble(args[0]);
31.         int type = Integer.parseInt(args[1]);
32.         double x = Double.parseDouble(args[2]);
33.         double y = Double.parseDouble(args[3]);
34.         double z = Double.parseDouble(args[4]);
35.
36.         Frame f = new Frame(max);
37.         f.addShape(type, x, y, z);
38.         System.out.println(f+"\narea: "+f.getArea());
39.     }
40. }
```

java Frame 10 1 0 0 15

java Frame 10 2 0 0 25

A test case that can improve the all-du-pairs coverage?

	getArea	getArea	getArea
def			
use	maxX, maxY, shape	side	radius

	adjust	adjust
def	x, y, side	x, y, radius
use	side	radius

+ constructors
(define everything)

```
9. class Frame {  
  
10.     private Shape shape;  
11.     private double maxX, maxY;  
...  
25.     public void addShape(int type, double x, double y, double z) {  
26.         if (type==1) {  
27.             shape = new Circle(x, y, z);  
28.         } else {  
29.             shape = new Square(x, y, z);  
30.         }  
31.  
32.         if (shape.getMaxX()>this.maxX || shape.getMaxY() > this.maxY) {  
33.             shape.adjust(this.maxX, this.maxY);  
34.         }  
35.     }  
...  
44.     public static void main(String[] args) {  
45.         double max = Double.parseDouble(args[0]);  
46.         int type = Integer.parseInt(args[1]);  
47.         double x = Double.parseDouble(args[2]);  
48.         double y = Double.parseDouble(args[3]);  
49.         double z = Double.parseDouble(args[4]);  
  
50.         Frame f = new Frame(max);  
51.         f.addShape(type, x, y, z);  
52.         System.out.println(f+"\narea: "+f.getArea());  
53.     }  
54. }
```

Test evaluation

- The sequence of statements or inputs can be coded, but how do we know if the obtained output is correct?
- Tests can include assertions that ensure that some properties hold.
- A programmer/tester looks at it
 - program specifications can be used to verify that the program behaves as expected.
 - Formal specifications simplify the comparison of expected and actual behavior, but they are costly.
 - Internal values of the program can be exposed for a tester to determine if the program behaves as expected (e.g., using jdb).

Best practices

- Create plenty of unit tests for the methods.
- Write these tests using a framework (e.g., JUnit).
- Create a selection of integration tests that run below the user interface.
- Create a few end-to-end tests that exercise the user interface.
- Re-execute tests often.
- Automate testing! (GitHub actions run tests at commit to main branch).